

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG

-----□□□-----



BÁO CÁO THỰC NGHIỆM

Học phần: An toàn & Bảo mật thông tin

**Chủ đề: Xây dựng chương trình mã hóa và giải mã
RSA (sử dụng ngôn ngữ Java,C#)**

Giáo viên hướng dẫn : TS. Phạm Văn Hiệp

Nhóm sinh viên thực hiện :

- | | |
|---------------------|-------------------|
| 1. Bùi Tiến Mạnh | Mã SV: 2024606502 |
| 2. Chu Thái Bảo | Mã SV: 2024604494 |
| 3. Nguyễn Thị Hương | Mã SV: 2024606807 |
| 4. Nguyễn Tiến Lợi | Mã SV: 2024604576 |
| 5. Nguyễn Hồng Sơn | Mã SV: 2024601597 |

Mã Lớp học phần: 20252IT6001001

Nhóm: 5

Hà Nội - Năm 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN.....	1
1.1. Tổng quan về An toàn bảo mật thông tin.....	1
1.1.1. Khái niệm về bảo mật dữ liệu và sự cần thiết của mã hóa.....	1
1.1.2. Các mục tiêu cơ bản.....	2
1.1.3. Phân loại hệ mật mã.....	4
1.2. Các kiến thức cơ sở.....	6
1.2.1. Kiến thức toán học.....	7
1.2.2. Kiến thức lập trình.....	10
1.3. Nội dung nghiên cứu.....	11
1.3.1. Lý do chọn đề tài.....	11
1.3.2. Nội dung nghiên cứu chính.....	13
1.3.3. Phạm vi nghiên cứu.....	15
1.3.4. Phương pháp nghiên cứu và triển khai hệ thống.....	17
CHƯƠNG 2: KẾT QUẢ NGHIÊN CỨU.....	20
2.1. Nghiên cứu hệ mã hóa khóa công khai.....	20
2.1.1. Nguyên lý hoạt động của hệ mật mã khóa công khai.....	20
2.1.2. Quy trình trao đổi khóa và vấn đề phân phối khóa.....	22
2.1.3. So sánh ưu, nhược điểm giữa mã hóa khóa công khai và khóa bí mật.....	24
2.2. Tìm hiểu chi tiết thuật toán RSA.....	24
2.2.1. Lịch sử hình thành và cơ sở lý thuyết của RSA.....	24
2.2.2. Quy trình thuật toán.....	24
2.2.3. Các cuộc tấn công phổ biến vào RSA và cách phòng chống.....	24
2.3. Thiết kế chương trình và cài đặt thuật toán.....	24
2.3.1. Thiết kế kịch bản chương trình.....	24
2.3.2. Giới thiệu ngôn ngữ lập trình sử dụng.....	24
2.3.3. Cài đặt thuật toán với ngôn ngữ JAVA.....	24
2.3.4. Cài đặt thuật toán với ngôn ngữ C#.....	24
CHƯƠNG 3: KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM.....	24
3.1. Kiến thức và kỹ năng đạt được.....	24
3.2. Bài học kinh nghiệm.....	24
3.3. Đề xuất và hướng phát triển.....	24

CHƯƠNG 1: TỔNG QUAN

1.1. Tổng quan về An toàn bảo mật thông tin.

Trong thời đại công nghệ thông tin phát triển mạnh mẽ, dữ liệu đóng vai trò vô cùng quan trọng đối với cá nhân, tổ chức và doanh nghiệp. Các thông tin như tài khoản người dùng, dữ liệu khách hàng, thông tin tài chính, hồ sơ cá nhân hay tài liệu nội bộ đều được lưu trữ và trao đổi thông qua hệ thống máy tính và mạng Internet. Vì vậy, việc bảo vệ dữ liệu khỏi các nguy cơ bị đánh cắp, thay đổi hoặc truy cập trái phép trở thành yêu cầu thiết yếu trong mọi hệ thống thông tin.

1.1.1. Khái niệm về bảo mật dữ liệu và sự cần thiết của mã hóa.

1. Khái niệm về bảo mật dữ liệu

Bảo mật dữ liệu là tập hợp các biện pháp, kỹ thuật và quy trình được áp dụng nhằm bảo vệ dữ liệu (ở trạng thái lưu trữ, đang xử lý hoặc đang truyền tải) khỏi bị truy cập, sử dụng, tiết lộ, sửa đổi, sao chép hoặc phá hủy trái phép.

Trong an toàn thông tin, bảo mật dữ liệu luôn hướng tới việc đảm bảo 3 mục tiêu cốt lõi (được gọi là Mô hình CIA Triad):

-**Tính bảo mật (Confidentiality):** Đảm bảo rằng dữ liệu chỉ có những người được cấp quyền mới có thể tiếp cận và đọc được. Người không có thẩm quyền dù có lấy được dữ liệu cũng không thể hiểu được nội dung.

-**Tính toàn vẹn (Integrity):** Đảm bảo dữ liệu không bị thay đổi, sửa đổi hoặc xóa bỏ một cách trái phép trong quá trình lưu trữ hay truyền tải. Nếu có sự thay đổi, hệ thống phải phát hiện ra ngay.

-**Tính sẵn sàng (Availability):** Đảm bảo dữ liệu luôn sẵn sàng phục vụ người dùng có thẩm quyền bất cứ khi nào họ có nhu cầu truy cập.

2. Sự cần thiết của mã hóa

Mã hóa là quá trình biến đổi dữ liệu thành các ký tự ngẫu nhiên không thể đọc được, chỉ có thể giải mã bằng khóa mật mã. Nó vô cùng cần thiết để bảo vệ thông tin nhạy cảm khỏi bị đánh cắp, đảm bảo quyền riêng tư, xác thực nguồn gốc và chống giả mạo.

Bảo vệ dữ liệu khỏi truy cập trái phép: Dù lưu trữ trên thiết bị (như laptop, ổ cứng) hay truyền tải qua Internet, dữ liệu của bạn luôn là mục tiêu của tin tặc. Mã hóa chuyển đổi dữ liệu gốc thành văn bản mã (ciphertext), khiến tin tặc không thể đọc hoặc sử dụng được thông tin ngay cả khi đánh cắp thành công.

Đảm bảo quyền riêng tư các nhân: Mã hóa bảo vệ các thông tin cá nhân của bạn như tin nhắn, email, thông tin tài khoản ngân hàng, hoặc dữ liệu y tế,...

khỏi sự giám sát hoặc nghe lén từ các bên thứ ba, cơ quan không có thẩm quyền hoặc các dịch vụ độc hại.

Đảm bảo toàn vẹn thông tin: Mã hóa đóng vai trò như một lớp khóa niêm phong. Khi truyền dữ liệu qua mạng, mọi hành vi can thiệp, chỉnh sửa hay giả mạo dữ liệu mà không thông qua giải mã hợp lệ sẽ ngay lập tức bị phát hiện hoặc làm cho tệp tin trở nên vô nghĩa.

Xác thực và tạo dựng lòng tin: Mã hóa đóng vai trò như một lớp khóa niêm phong. Khi truyền dữ liệu qua mạng, mọi hành vi can thiệp, chỉnh sửa hay giả mạo dữ liệu mà không thông qua giải mã hợp lệ sẽ ngay lập tức bị phát hiện hoặc làm cho tệp tin trở nên vô nghĩa.

Tuân thủ các tiêu chuẩn pháp lý: Trong nhiều lĩnh vực như tài chính, y tế hay giáo dục, các tổ chức luật pháp yêu cầu doanh nghiệp bắt buộc phải mã hóa dữ liệu khách hàng. Việc tuân thủ này không chỉ tránh các khoản phạt nặng mà còn xây dựng lòng tin từ đối tác và người dùng.

1.1.2. Các mục tiêu cơ bản.

Trong an toàn và bảo mật thông tin, việc bảo vệ dữ liệu không chỉ nhằm ngăn chặn người khác đọc trộm thông tin, mà còn phải bảo đảm dữ liệu không bị thay đổi trái phép và xác định đúng chủ thể tham gia trao đổi thông tin. Vì vậy, các mục tiêu cơ bản của an toàn bảo mật thông tin thường được thể hiện qua ba yếu tố quan trọng: tính bảo mật, tính toàn vẹn và tính xác thực.

1. Tính bảo mật

Tính bảo mật là khả năng bảo vệ thông tin khỏi sự truy cập, khai thác hoặc tiết lộ trái phép. Nói cách khác, thông tin chỉ được phép hiển thị hoặc sử dụng bởi những người, hệ thống hoặc đối tượng có quyền hợp lệ. Đây là một trong những yêu cầu quan trọng nhất trong bảo vệ dữ liệu, đặc biệt đối với các thông tin nhạy cảm như tài khoản đăng nhập, dữ liệu khách hàng, thông tin tài chính, hợp đồng, hồ sơ cá nhân hoặc bí mật kinh doanh.

Trong môi trường mạng, dữ liệu có thể bị đánh cắp thông qua nhiều hình thức như nghe trộm đường truyền, truy cập trái phép vào máy chủ, khai thác lỗ hổng phần mềm hoặc giả mạo người dùng hợp pháp. Khi tính bảo mật không được bảo đảm, thông tin có thể bị lộ ra ngoài, gây ảnh hưởng nghiêm trọng đến cá nhân, tổ chức hoặc doanh nghiệp. Hậu quả có thể bao gồm mất uy tín, thiệt hại tài chính, rò rỉ dữ liệu riêng tư và làm suy giảm niềm tin của người sử dụng.

Để bảo đảm tính bảo mật, các hệ thống thường sử dụng nhiều biện pháp khác nhau như mã hóa dữ liệu, phân quyền truy cập, xác thực người dùng, sử dụng mật khẩu mạnh, bảo vệ khóa bí mật và kiểm soát an toàn trên đường truyền. Trong đó, mã hóa là một kỹ thuật quan trọng, giúp biến đổi dữ liệu ban đầu thành dạng không thể hiểu được đối với người không có khóa giải mã phù hợp.

2. Tính toàn vẹn

Tính toàn vẹn là khả năng bảo đảm thông tin không bị thay đổi, chỉnh sửa, thêm, xóa hoặc phá hoại một cách trái phép trong quá trình lưu trữ, xử lý và truyền tải. Một hệ thống bảo mật tốt không chỉ cần giữ kín thông tin, mà còn phải bảo đảm rằng thông tin khi đến tay người nhận vẫn giữ nguyên nội dung như ban đầu.

Trong thực tế, tính toàn vẹn có vai trò rất quan trọng đối với các hệ thống giao dịch điện tử, ngân hàng, thương mại điện tử, quản lý sinh viên, hồ sơ bệnh án, văn bản hành chính và các hệ thống cơ sở dữ liệu. Ví dụ, nếu một giao dịch chuyển tiền bị thay đổi số tiền hoặc tài khoản nhận trong quá trình xử lý mà hệ thống không phát hiện được, hậu quả có thể rất nghiêm trọng. Tương tự, nếu dữ liệu điểm số, hồ sơ cá nhân hoặc hợp đồng điện tử bị chỉnh sửa trái phép thì tính chính xác và độ tin cậy của hệ thống sẽ bị ảnh hưởng.

Để bảo đảm tính toàn vẹn, người ta thường sử dụng các kỹ thuật như hàm băm, mã kiểm tra toàn vẹn, mã xác thực thông báo và chữ ký số. Các kỹ thuật này giúp phát hiện việc dữ liệu đã bị thay đổi hay chưa. Trong các hệ thống sử dụng RSA, tính toàn vẹn thường được kết hợp với hàm băm và chữ ký số để kiểm tra xem thông điệp có còn nguyên vẹn sau khi truyền hay không.

3. Tính xác thực

Tính xác thực là khả năng kiểm tra và xác nhận danh tính của các đối tượng tham gia vào quá trình trao đổi thông tin. Mục tiêu của tính xác thực là bảo đảm rằng người gửi, người nhận hoặc hệ thống đang giao tiếp thực sự là đối tượng đã được khai báo, không phải kẻ giả mạo.

Trong môi trường mạng, nguy cơ giả mạo danh tính luôn tồn tại. Kẻ tấn công có thể giả danh người dùng hợp pháp để đăng nhập vào hệ thống, gửi thông điệp sai lệch, thực hiện giao dịch trái phép hoặc đánh lừa người nhận. Vì vậy, tính xác thực giúp hệ thống kiểm tra đúng danh tính của người dùng, nguồn gốc của thông điệp và độ tin cậy của quá trình trao đổi dữ liệu.

Các phương pháp xác thực phổ biến gồm tên đăng nhập và mật khẩu, mã OTP, chứng chỉ số, chữ ký số, xác thực đa yếu tố và các cơ chế xác thực dựa trên khóa mật mã. Đối với thuật toán RSA, tính xác thực được thể hiện rõ trong ứng dụng chữ ký số. Người gửi sử dụng khóa riêng để tạo chữ ký, còn người nhận dùng khóa công khai tương ứng để kiểm tra chữ ký đó. Nếu chữ ký hợp lệ, người nhận có thể tin rằng thông điệp được gửi từ đúng chủ thể và không phải từ một đối tượng giả mạo.

4. Mối quan hệ giữa các mục tiêu

Ba mục tiêu trên có mối quan hệ chặt chẽ với nhau trong việc bảo vệ hệ thống thông tin. Tính bảo mật giúp dữ liệu không bị lộ cho người không có quyền. Tính toàn vẹn giúp dữ liệu không bị sửa đổi trái phép. Tính xác thực giúp xác định đúng danh tính của các bên tham gia trao đổi thông tin.

Một hệ thống chỉ được xem là an toàn khi kết hợp đồng thời các yếu tố này. Nếu dữ liệu được mã hóa nhưng không kiểm tra được người gửi là ai, hệ thống vẫn có thể bị giả mạo. Nếu xác định đúng người gửi nhưng dữ liệu bị thay đổi trong quá trình truyền, thông tin nhận được cũng không còn đáng tin cậy. Do đó, bảo mật thông tin cần được xây dựng dựa trên sự phối hợp giữa bảo mật, toàn vẹn và xác thực.

Đối với đề tài nghiên cứu về hệ mật RSA, ba mục tiêu này là cơ sở quan trọng để tìm hiểu vai trò của mã hóa khóa công khai. RSA có thể được sử dụng để bảo vệ dữ liệu thông qua mã hóa, hỗ trợ xác thực thông qua chữ ký số và kết hợp với hàm băm để kiểm tra tính toàn vẹn của thông tin. Vì vậy, việc nắm rõ các mục tiêu cơ bản của an toàn bảo mật thông tin là nền tảng cần thiết trước khi nghiên cứu sâu hơn về thuật toán RSA.

1.1.3. Phân loại hệ mật mã

1. Hệ mã khoá bí mật

Mật mã khoá bí mật hay còn gọi là mật mã đối xứng ra đời từ rất sớm. Từ khi máy tính chưa ra đời, mật mã khoá bí mật đã đóng vai trò quan trọng trong việc mã hoá thông tin. Mật mã khoá bí mật yêu cầu người gửi và người nhận phải thỏa thuận một khóa trước khi thông báo được gửi, và khóa này phải được cất giữ bí mật. Độ an toàn của thuật toán này vẫn phụ thuộc vào khóa, nếu để lộ ra khóa này nghĩa là bất kì người nào cũng có thể mã hóa và giải mã hệ thống mật mã.

Tại nơi gửi (nguồn thông báo) có một bản rõ R được sinh ra. Để mã R cần có một khoá K. Nếu K được sinh tại nguồn thông báo thì nó phải được chuyển tới đích thông báo theo một kênh an toàn. Hoặc một bên thứ ba có thể sinh khoá và chuyển một cách an toàn tới cả nguồn và đích.

Với thông báo R và khoá mã K, thuật toán mã E sẽ tạo ra bản mã

$$M = E_k(R)$$

Tại nơi nhận (đích thông báo) với bản mã M và khoá mã K, thuật toán giải mã D sẽ tạo ra bản rõ $R = D_K(M)$.

Một kẻ tấn công thu được M nhưng không có khoá K, anh ta phải cố gắng khôi phục R hoặc khoá K. Thừa nhận rằng kẻ tấn công biết thuật toán mã E và thuật toán giải mã D. Nếu kẻ tấn công chỉ quan tâm đến nội dung thông báo, họ cố khôi phục R bằng việc sinh ra một ước lượng R' của R. Tuy nhiên thường kẻ tấn công mong muốn tìm ra khoá K để giải mã các thông báo tiếp theo, bằng cách sinh ra một khoá ước lượng K' của K. Độ bảo mật của mật mã khóa bí mật là thước đo mức độ khó khăn của việc tìm ra thông báo rõ hoặc khoá khi biết bản mã.

Ưu điểm của mật mã khoá bí mật là tốc độ mã hoá và giải mã nhanh.

Tuy nhiên, mật mã khoá bí mật lại có khá nhiều nhược điểm:

- Cần có kênh an toàn để trao đổi khoá.
- Trên môi trường mạng có N người dùng, thì cần $N(N-1)/2$ khoá để $N(N-1)/2$ cặp người trao đổi thông tin. Như vậy, sẽ cần nhiều khoá, khi đó việc quản lý khóa phức tạp.
- Không thể thiết lập được chữ ký điện tử.

2. Hệ mã khoá công khai

Năm 1976, Diffie và Hellman đưa ra phương pháp mã hoá có thể giải quyết các nhược điểm của mã khoá bí mật. Đó là mã hoá khoá công khai (PKC-Public key Cryptography) hay còn gọi là mã hoá bất đối xứng (Asymmetric Cryptography).

Nguyên tắc của mã hoá khoá công khai:

- Mỗi người dùng có 2 khoá: một khoá bí mật (K_S) và một khoá công khai (K_P).
- Khoá thứ nhất được sử dụng để giải mã, còn khoá thứ hai được dùng để tạo mã (mã hoá).

Trong hệ mã khoá công khai, việc phân phối khoá sẽ dễ dàng hơn qua các kênh cung cấp khoá công cộng, số lượng khoá hệ thống quản lý cũng sẽ ít hơn (n người dùng cần n khoá).

Các dịch vụ được xây dựng dựa trên các hệ mã khoá PKC: chữ ký điện tử, thỏa thuận trao đổi khóa, ...

Các yêu cầu của loại hệ mã hoá PKC:

- Dễ dàng tạo ra một cặp khoá K_S , K_P
- Dễ dàng mã hoá M để tạo ra bản mã: $C=E(K_P, M)$
- Dễ dàng giải mã C để phục hồi M: $M=D(K_S, C)$
- Kẻ tấn công không thể xác định được K_S dù biết K_P
- Kẻ tấn công dù biết K_P và bản mã C nhưng không khôi phục lại được M.
- Hai khoá K_S và K_P có thể áp dụng theo thứ tự tùy ý trong trường hợp mã hoá hoặc giải mã.

An toàn của mã khoá công khai:

- Độ an toàn của thuật toán mã hoá công khai phụ thuộc vào độ khó của bài toán ngược.
- Thăm mã bằng phương pháp vét cạn về mặt lý thuyết là có thể thực hiện được. Tuy nhiên, trên thực tế các khoá sử dụng lớn (>512 bit) sẽ rất khó thực hiện vét cạn.
- Khi sử dụng các khoá rất lớn ($>>512$ bit) có thể chống lại được một số phương pháp thăm mã tiên tiến.
- Quá trình cài đặt thuật toán khoá công khai chậm hơn nhiều so với thuật toán khoá bí mật.

Nhận xét về hệ mã khoá công khai:

- Một hệ thống mã hoá PKC có thể đáp ứng hai mục đích:
 - + Bảo mật thông tin và truyền tin
 - + Chứng thực và chữ ký điện tử
- Hai thuật toán đáp ứng các ứng dụng trên thành công nhất là RSA và Elgamal.
- Hệ mã khoá công khai PKC có tốc độ thực hiện chậm nhưng lại phù hợp khi áp dụng trong các ứng dụng cần có tính an toàn cao.

1.2. Các kiến thức cơ sở

1.2.1. Kiến thức toán học:

1. Số nguyên tố và bài toán phân tích thừa số nguyên

Số nguyên tố là số nguyên dương lớn hơn 1, chỉ có hai ước số dương là 1 và chính nó. Trong hệ mật mã RSA, độ an toàn chủ yếu dựa trên độ khó của bài toán phân tích thừa số nguyên tố (thường ký hiệu là p và q). Mật mã RSA dựa trên thực tế toán học: việc nhân hai số nguyên tố lớn để tạo ra module n :

$$n = p \times q$$

thì rất dễ và nhanh chóng, nhưng bài toán ngược lại phân tích số n ra thừa số nguyên tố để tìm lại p và q là một bài toán cực kỳ phức tạp và chưa tồn tại thuật toán thời gian đa thức khả thi trên máy tính cổ điển đối với các số có kích thước đủ lớn.

2. Số học Mô-đun và Kỹ thuật Lũy thừa Module

Các tính chất đại số của số học module: Phép toán module tuân theo các quy tắc cấu trúc đại số của vành số nguyên đồng dư. Định lý cơ bản của số học module cho phép thực hiện các phép toán cộng, trừ, nhân trước hoặc sau khi lấy module mà không làm thay đổi kết quả cuối cùng:

$$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$$

$$(a - b) \bmod n = ((a \bmod n) - (b \bmod n)) \bmod n$$

$$(a \times b) \bmod n = ((a \bmod n) \times (b \bmod n)) \bmod n$$

Tính chất này cho phép máy tính xử lý các số hạng trung gian nhỏ hơn rất nhiều, tránh hiện tượng tràn dữ liệu (overflow).

Bài toán bùng nổ số mũ và Thuật toán Lũy thừa Module: Trong hệ mật mã RSA, quy trình mã hóa đòi hỏi tính toán biểu thức có dạng:

$$C = M^e \pmod n$$

Khi độ dài của khóa e lên tới 2048-bit, giá trị của M^e là một số khổng lồ. Hệ thống áp dụng thuật toán Bình phương và Nhân (Square-and-Multiply). Thuật toán này phân tích số mũ e dưới dạng biểu diễn nhị phân:

$$e = (e_k e_{(k-1)} \dots e_1 e_0)_2$$

Quy trình thực hiện: Khởi tạo biến tích lũy $A = 1$. Duyệt qua từng bit của số mũ từ bit cao nhất đến thấp nhất. Với mỗi bước, thực hiện bình phương biến tích lũy $A = (A \times A) \bmod n$. Nếu bit hiện tại bằng 1, thực hiện phép nhân bổ sung $A =$

$(A \times M) \bmod n$. Kết quả thu được chính là giá trị C . Nhờ cơ chế này, độ phức tạp tính toán giảm từ hàm mũ xuống hàm logarit, giúp máy tính xử lý phép toán chỉ trong vài phần nghìn giây.

3. Ước chung lớn nhất và Nghịch đảo Module

Khái niệm Nghịch đảo Modulo: Trong trường số học module, không tồn tại khái niệm phân số. Nghịch đảo mô-đun của một số nguyên a theo mô-đun n được định nghĩa là một số nguyên x thỏa mãn phương trình đồng dư:

$$a \times x \equiv 1 \pmod{n}$$

Điều kiện cần và đủ để tồn tại số nghịch đảo x là a và n phải là hai số nguyên tố cùng nhau, tức là ước chung lớn nhất:

$$\gcd(a, n) = 1$$

Trong RSA, khóa bí mật d chính là nghịch đảo modulo của khóa công khai e theo mô-đun phi hàm số Euler của n :

$$e \times d \equiv 1 \pmod{(p-1) \times (q-1)}$$

Ứng dụng toán học Euclid mở rộng: Để tìm nghịch đảo module d , hệ thống sử dụng toán học Euclid mở rộng dựa trên hệ thức Bézout, chỉ ra rằng luôn tồn tại hai số nguyên x và y sao cho:

$$a \times x + n \times y = \gcd(a, n)$$

Vì ước chung lớn nhất bằng 1, phương trình chuyển thành:

$$e \times x + \phi(n) \times y = 1$$

Khi lấy mô-đun $\phi(n)$ cho cả hai vế, ta thu được:

$$e \times x \equiv 1 \pmod{\phi(n)}$$

Do đó, giá trị x tìm được chính là khóa bí mật d cần tìm.

4. Phi hàm Euler (Euler's Totient Function)

Phi hàm Euler, ký hiệu là $\phi(n)$, biểu diễn số lượng các số nguyên dương nhỏ hơn n và nguyên tố cùng nhau với n . Đối với một số nguyên tố p bất kỳ, ta có:

$$\phi(p) = p - 1$$

Trong thuật toán RSA, do $n = p \times q$ (với p và q là hai số nguyên tố phân biệt), áp dụng tính chất hàm nhân tính ta có công thức tính nhanh:

$$\phi(n) = (p - 1) \times (q - 1)$$

Giá trị này đóng vai trò cốt lõi trong bước sinh khóa, được sử dụng làm cơ sở để tính toán khóa bí mật thông qua phương trình đồng dư bất buộc:

$$e \times d \equiv 1 \pmod{\phi(n)}$$

5. Định lý Fermat nhỏ và ứng dụng trong kiểm tra số nguyên tố

Phát biểu định lý: Định lý Fermat nhỏ phát biểu rằng nếu p là một số nguyên tố và a là một số nguyên không chia hết cho p , thì ta luôn có:

$$a^{(p-1)} \equiv 1 \pmod{p}$$

Nhân cả hai vế của phương trình với a , ta thu được một hệ quả tổng quát hơn:

$$a^p \equiv a \pmod{p}$$

Ứng dụng trong việc sinh số nguyên tố cho hệ mật RSA: Định lý Fermat nhỏ đóng vai trò cốt lõi trong các thuật toán kiểm tra tính nguyên tố (Primality Testing) như Thuật toán Miller-Rabin. Thay vì phân tích ước số theo cách truyền thống, máy tính sẽ chọn một số cơ sở a ngẫu nhiên và kiểm tra xem biểu thức trên có thỏa mãn hay không. Nếu phương trình sai, hệ thống khẳng định 100% p là hợp số. Qua nhiều vòng thử nghiệm, xác suất xác định đúng số nguyên tố tiến rất gần tới 1.

6. Định lý phần dư Trung Hoa và tối ưu hóa hiệu năng (CRT)

Phát biểu toán học của định lý: Định lý phần dư Trung Hoa (CRT) phát biểu rằng nếu các số nguyên $m_1, m_2, \dots, m_k$ nguyên tố cùng nhau đôi một, thì hệ phương trình đồng dư bậc nhất:

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad x \equiv a_k \pmod{m_k}$$

Luôn luôn có một nghiệm duy nhất x theo module

$$M = m_1 \times m_2 \times \dots \times m_k$$

Ứng dụng tăng tốc độ giải mã mã hóa RSA (RSA-CRT): Giai đoạn giải mã RSA yêu cầu tính toán biểu thức:

$$M = C^d \pmod{n}$$

Vì d và n là các số cực kỳ lớn, CRT được áp dụng để phân rã phép toán trên module lớn n thành hai phép toán trên các module nhỏ hơn là p và q . Hệ thống sẽ tính trước các giá trị trung gian cố định:

$$d_p = d \bmod (p - 1), d_q = d \bmod (q - 1), q_{(inv)} = q^{(-1)} \bmod p$$

Khi thực hiện giải mã, thiết bị sẽ tính hai bản rõ thành phần trên module p và q:

$$M_p = C^{(d_p)} \bmod p, \quad M_q = C^{(d_q)} \bmod q$$

Sau đó, áp dụng công thức phối hợp phức hồi của Garner để tìm lại bản rõ gốc M:

$$h = (q_{(inv)} \times (M_p - M_q)) \bmod p$$

$$M = M_q + h \times q$$

Việc tính toán phân tách trên cấu trúc này giúp giảm khối lượng tính toán lý thuyết xuống khoảng 4 lần.

1.2.2. Kiến thức lập trình

Để xây dựng chương trình mã hóa và giải mã RSA, nhóm sử dụng ngôn ngữ lập trình Java và C#. Đây là các ngôn ngữ lập trình hướng đối tượng phổ biến, có khả năng xử lý dữ liệu tốt, hỗ trợ nhiều thư viện liên quan đến bảo mật và mã hóa thông tin. Việc sử dụng Java và C# giúp chương trình dễ xây dựng, dễ kiểm thử và thuận lợi cho việc phát triển các chức năng của hệ mật mã RSA.

Trong quá trình thực hiện đề tài, nhóm sử dụng các kiến thức lập trình cơ bản như biến, kiểu dữ liệu, toán tử, cấu trúc điều khiển và hàm để xây dựng thuật toán mã hóa và giải mã dữ liệu. Các cấu trúc điều kiện if...else và vòng lặp for, while được sử dụng để kiểm tra điều kiện, thực hiện các phép tính lặp và xử lý dữ liệu trong chương trình.

Ngoài ra, chương trình còn sử dụng kiến thức về lập trình hướng đối tượng nhằm tổ chức mã nguồn thành các lớp và các phương thức riêng biệt. Mỗi chức năng như sinh khóa, mã hóa, giải mã hoặc kiểm tra số nguyên tố đều được xây dựng thành các hàm độc lập, giúp chương trình dễ quản lý, dễ sửa lỗi và thuận tiện cho việc nâng cấp sau này.

Do thuật toán RSA sử dụng các số nguyên có kích thước rất lớn nên các kiểu dữ liệu thông thường không đáp ứng được yêu cầu tính toán. Vì vậy chương trình sử dụng lớp BigInteger trong Java và C# để thực hiện các phép toán với số lớn như:

- Tính lũy thừa modulo
- Tính nghịch đảo modulo
- Phép chia lấy dư
- Phép nhân số nguyên lớn

Trong quá trình xây dựng chương trình, nhóm sử dụng một số thư viện hỗ trợ như:

- Đối với Java:
 - + `java.math.BigInteger`: hỗ trợ xử lý số nguyên lớn.
 - + `java.security.SecureRandom`: hỗ trợ sinh số ngẫu nhiên phục vụ quá trình tạo khóa RSA.
- Đối với C#:
 - + `System.Numerics.BigInteger`: hỗ trợ xử lý số nguyên lớn.
 - + `System.Security.Cryptography`: hỗ trợ các chức năng liên quan đến mã hóa và bảo mật dữ liệu.

Ngoài phần xử lý thuật toán, chương trình còn sử dụng kiến thức về xử lý chuỗi để nhập dữ liệu từ bàn phím, chuyển đổi ký tự sang dạng số và hiển thị kết quả mã hóa hoặc giải mã cho người dùng. Các thao tác này giúp chương trình có thể xử lý dữ liệu văn bản một cách chính xác và thuận tiện hơn trong quá trình sử dụng.

Việc áp dụng các kiến thức lập trình trên giúp chương trình thực hiện đầy đủ các chức năng của hệ mật mã RSA như sinh khóa công khai và khóa bí mật, mã hóa dữ liệu và giải mã dữ liệu. Đồng thời, quá trình xây dựng chương trình cũng giúp nâng cao kỹ năng lập trình và hiểu rõ hơn về nguyên lý hoạt động của hệ mật mã bất đối xứng trong an toàn và bảo mật thông tin.

1.3. Nội dung nghiên cứu:

1.3.1. Lý do chọn đề tài:

Trong lĩnh vực an toàn và bảo mật thông tin, thuật toán RSA không chỉ được sử dụng để mã hóa dữ liệu mà còn đóng vai trò rất quan trọng trong chữ ký số và bảo mật các giao dịch trực tuyến. Nhờ cơ chế sử dụng cặp khóa công khai và khóa bí mật, RSA giúp bảo đảm tính bảo mật, tính toàn vẹn và tính xác thực của thông tin trong môi trường mạng Internet.

1. Vai trò của RSA trong chữ ký số

Chữ ký số là kỹ thuật dùng để xác minh danh tính của người gửi và kiểm tra tính toàn vẹn của dữ liệu điện tử. Trong hệ thống chữ ký số RSA, người gửi sử dụng khóa bí mật để tạo chữ ký, còn người nhận sử dụng khóa công khai tương ứng để kiểm tra tính hợp lệ của chữ ký đó.

Quy trình hoạt động của chữ ký số RSA được thực hiện như sau:

- Người gửi tạo giá trị băm (hash) của thông điệp bằng các thuật toán băm như SHA-256.
- Giá trị băm này được mã hóa bằng khóa bí mật RSA để tạo thành chữ ký số.
- Người gửi gửi thông điệp kèm chữ ký số cho người nhận.
- Người nhận sử dụng khóa công khai của người gửi để giải mã chữ ký số và thu được giá trị băm ban đầu.
- Đồng thời, người nhận cũng tự tính giá trị băm của thông điệp nhận được.
- Nếu hai giá trị băm giống nhau, thông điệp được xác nhận là hợp lệ và chưa bị thay đổi.

Nhờ cơ chế trên, RSA giúp đảm bảo:

- Tính xác thực:** xác minh đúng người gửi thông điệp.
- Tính toàn vẹn:** phát hiện dữ liệu bị thay đổi trong quá trình truyền.
- Tính chống chối bỏ:** người gửi không thể phủ nhận việc đã gửi thông điệp vì chữ ký được tạo bằng khóa bí mật riêng.

Hiện nay, chữ ký số RSA được ứng dụng rộng rãi trong:

- Ký văn bản điện tử
- Hợp đồng điện tử
- Hóa đơn điện tử
- Chứng thư số
- Giao dịch ngân hàng trực tuyến
- Hệ thống chính phủ điện tử

2. Vai trò của RSA trong bảo mật giao dịch trực tuyến

Trong các giao dịch trực tuyến như thanh toán điện tử, thương mại điện tử hoặc ngân hàng trực tuyến, dữ liệu thường được truyền qua môi trường Internet công

cộng nên rất dễ bị nghe lén hoặc đánh cắp. RSA được sử dụng để bảo vệ quá trình trao đổi dữ liệu và xác thực các bên tham gia giao dịch.

Khi người dùng truy cập vào một website bảo mật sử dụng giao thức HTTPS, RSA thường được dùng trong quá trình thiết lập kết nối an toàn SSL/TLS. Máy chủ sẽ cung cấp khóa công khai thông qua chứng chỉ số, còn trình duyệt của người dùng sử dụng khóa công khai này để mã hóa dữ liệu hoặc trao đổi khóa phiên an toàn.

Vai trò của RSA trong giao dịch trực tuyến gồm:

- Mã hóa dữ liệu nhạy cảm như mật khẩu, thông tin tài khoản và dữ liệu thanh toán.
- Xác thực danh tính máy chủ nhằm chống giả mạo website.
- Hỗ trợ trao đổi khóa an toàn để thiết lập kết nối bảo mật.
- Bảo vệ dữ liệu khỏi nguy cơ nghe lén và tấn công trên đường truyền.

Trong thực tế, RSA thường không dùng để mã hóa toàn bộ dữ liệu vì tốc độ xử lý chậm hơn so với mã hóa đối xứng. Thay vào đó, RSA được sử dụng để trao đổi khóa bí mật, sau đó dữ liệu sẽ được mã hóa bằng các thuật toán đối xứng như AES nhằm tăng hiệu năng xử lý.

3. Ý nghĩa của RSA trong an toàn thông tin hiện đại

RSA là một trong những thuật toán mật mã bất đối xứng đầu tiên và có ảnh hưởng lớn trong lĩnh vực bảo mật hiện đại. Thuật toán này tạo nền tảng cho nhiều hệ thống bảo mật quan trọng trên Internet như: HTTPS, SSL/TLS, VPN, chữ ký số, chứng chỉ số, xác thực người dùng,...

Nhờ khả năng hỗ trợ mã hóa và xác thực, RSA góp phần xây dựng môi trường giao dịch điện tử an toàn, đáng tin cậy và bảo vệ dữ liệu trước các nguy cơ tấn công mạng. Vì vậy, RSA vẫn là một trong những thuật toán quan trọng và được ứng dụng rộng rãi trong các hệ thống an toàn thông tin hiện nay.

1.3.2. Nội dung nghiên cứu chính:

Mục tiêu nghiên cứu của đề tài là tìm hiểu hệ mật mã RSA và xây dựng chương trình thực hiện quá trình mã hóa, giải mã dữ liệu bằng thuật toán RSA trên hai ngôn ngữ lập trình Java và C#. Thông qua đề tài này, nhóm hướng đến việc nắm

vững cơ sở lý thuyết của hệ mật mã khóa công khai, hiểu rõ quy trình hoạt động của RSA và vận dụng kiến thức đã học để xây dựng chương trình minh họa thực tế.

Trước hết, đề tài tập trung nghiên cứu tổng quan về an toàn và bảo mật thông tin, đặc biệt là vai trò của mã hóa trong việc bảo vệ dữ liệu. Trong môi trường mạng, thông tin có thể bị nghe trộm, đánh cắp hoặc thay đổi trái phép trong quá trình truyền tải. Vì vậy, việc nghiên cứu một thuật toán mã hóa có khả năng bảo vệ dữ liệu như RSA là cần thiết, giúp làm rõ cách dữ liệu được chuyển đổi từ dạng bản rõ sang bản mã và khôi phục lại thông tin ban đầu thông qua quá trình giải mã.

Mục tiêu tiếp theo là tìm hiểu nguyên lý của hệ mật mã khóa công khai. RSA là một hệ mật mã bất đối xứng, sử dụng hai khóa khác nhau gồm khóa công khai và khóa bí mật. Khóa công khai được dùng để mã hóa dữ liệu, còn khóa bí mật được dùng để giải mã dữ liệu. Việc nghiên cứu cơ chế này giúp nhóm hiểu được sự khác biệt giữa mã hóa đối xứng và mã hóa bất đối xứng, đồng thời thấy được ưu điểm của RSA trong việc giải quyết vấn đề phân phối khóa khi trao đổi thông tin.

Đề tài cũng hướng đến việc nghiên cứu chi tiết quy trình của thuật toán RSA, bao gồm ba giai đoạn chính: tạo cặp khóa, mã hóa và giải mã. Ở giai đoạn tạo khóa, nhóm tìm hiểu cách chọn hai số nguyên tố, tính giá trị n , tính hàm Euler $\phi(n)$, chọn số mũ công khai e và tính số mũ bí mật d . Ở giai đoạn mã hóa, nhóm nghiên cứu cách sử dụng khóa công khai để biến đổi thông điệp ban đầu thành bản mã. Ở giai đoạn giải mã, nhóm tìm hiểu cách sử dụng khóa bí mật để khôi phục lại thông điệp gốc từ bản mã đã nhận.

Bên cạnh việc nghiên cứu lý thuyết, đề tài đặt mục tiêu xây dựng chương trình mã hóa và giải mã RSA bằng ngôn ngữ Java và C#. Chương trình cần mô phỏng được các bước cơ bản của thuật toán, bao gồm nhập dữ liệu đầu vào, tạo cặp khóa, mã hóa thông điệp và giải mã thông điệp. Việc triển khai trên hai ngôn ngữ lập trình giúp nhóm củng cố kỹ năng lập trình, đồng thời so sánh cách biểu diễn thuật toán RSA trong từng môi trường lập trình khác nhau.

Một mục tiêu quan trọng khác là kiểm tra tính đúng đắn của chương trình sau khi cài đặt. Kết quả mã hóa và giải mã cần bảo đảm rằng thông điệp sau khi giải mã phải trùng khớp với thông điệp ban đầu. Thông qua quá trình thử nghiệm, nhóm có thể đánh giá được chương trình đã thực hiện đúng quy trình thuật toán RSA hay chưa, đồng thời phát hiện và điều chỉnh các lỗi phát sinh trong quá trình lập trình.

Ngoài ra, đề tài còn hướng đến việc tìm hiểu độ an toàn của hệ mật RSA. Độ an toàn của RSA dựa trên độ khó của bài toán phân tích một số nguyên lớn thành tích của hai số nguyên tố. Nếu các tham số được lựa chọn đủ lớn và khóa bí mật được bảo vệ an toàn, việc suy ra khóa bí mật từ khóa công khai là rất khó thực hiện trong thời gian tính toán hợp lý. Qua đó, nhóm có thể hiểu rõ hơn vì sao RSA được sử dụng trong nhiều ứng dụng bảo mật như bảo vệ dữ liệu, xác thực và giao dịch trực tuyến.

Tóm lại, mục tiêu nghiên cứu của đề tài là nắm vững nguyên lý hoạt động của thuật toán RSA, hiểu được quá trình tạo khóa, mã hóa và giải mã, phân tích được cơ sở bảo mật của thuật toán, đồng thời xây dựng chương trình minh họa bằng Java và C#. Kết quả nghiên cứu giúp nhóm củng cố kiến thức về an toàn bảo mật thông tin, vận dụng cơ sở toán học vào lập trình và có cái nhìn thực tế hơn về ứng dụng của mật mã học trong bảo vệ dữ liệu.

1.3.3. Phạm vi nghiên cứu:

Phạm vi nghiên cứu của đề tài tập trung vào việc tìm hiểu hệ mật mã RSA và xây dựng chương trình mã hóa, giải mã dữ liệu sử dụng ngôn ngữ lập trình Java và C#. Đề tài được triển khai ở mức nghiên cứu lý thuyết kết hợp với xây dựng chương trình minh họa, nhằm làm rõ cách thức hoạt động cơ bản của thuật toán RSA trong quá trình bảo vệ thông tin.

Về phạm vi lý thuyết, đề tài tập trung nghiên cứu các kiến thức cơ bản liên quan đến an toàn và bảo mật thông tin, vai trò của mã hóa trong bảo vệ dữ liệu, phân loại hệ mật mã và nguyên lý của hệ mật mã khóa công khai. Trong đó, nội dung trọng tâm là hệ mật mã RSA, bao gồm cơ sở hình thành, đặc điểm của mã hóa bất đối xứng, mối quan hệ giữa khóa công khai và khóa bí mật, cũng như vai trò của RSA trong quá trình mã hóa và giải mã dữ liệu.

Đề tài cũng giới hạn phạm vi nghiên cứu vào các kiến thức toán học cần thiết để hiểu và triển khai RSA. Các nội dung toán học được sử dụng gồm số nguyên tố, phép toán modulo, ước chung lớn nhất, số nguyên tố cùng nhau, hàm Euler và nghịch đảo modulo. Đây là các kiến thức nền tảng để thực hiện các bước sinh khóa, mã hóa và giải mã trong hệ mật RSA.

Về phạm vi thuật toán, đề tài tập trung vào ba giai đoạn chính của RSA. Giai đoạn thứ nhất là tạo cặp khóa gồm khóa công khai và khóa bí mật. Giai đoạn thứ hai là mã hóa thông điệp ban đầu thành bản mã bằng khóa công khai. Giai đoạn thứ

ba là giải mã bản mã để khôi phục lại thông điệp ban đầu bằng khóa bí mật. Các công thức và bước xử lý được trình bày theo đúng quy trình cơ bản của thuật toán RSA.

Về phạm vi cài đặt, đề tài xây dựng chương trình minh họa thuật toán RSA bằng hai ngôn ngữ lập trình Java và C#. Chương trình tập trung vào các chức năng chính như nhập dữ liệu đầu vào, tạo khóa, mã hóa thông điệp, giải mã thông điệp và hiển thị kết quả. Việc triển khai trên hai ngôn ngữ giúp làm rõ cách biểu diễn thuật toán trong từng môi trường lập trình, đồng thời củng cố kỹ năng lập trình và xử lý số học trong bài toán mật mã.

Về phạm vi kiểm thử, đề tài thực hiện kiểm tra chương trình thông qua các trường hợp dữ liệu đơn giản nhằm đánh giá tính đúng đắn của thuật toán. Kết quả giải mã cần khôi phục đúng thông điệp ban đầu sau khi đã được mã hóa. Việc kiểm thử chủ yếu nhằm xác nhận chương trình hoạt động đúng theo quy trình RSA, chưa đi sâu vào kiểm thử hiệu năng, kiểm thử bảo mật chuyên sâu hoặc đánh giá trên hệ thống thực tế quy mô lớn.

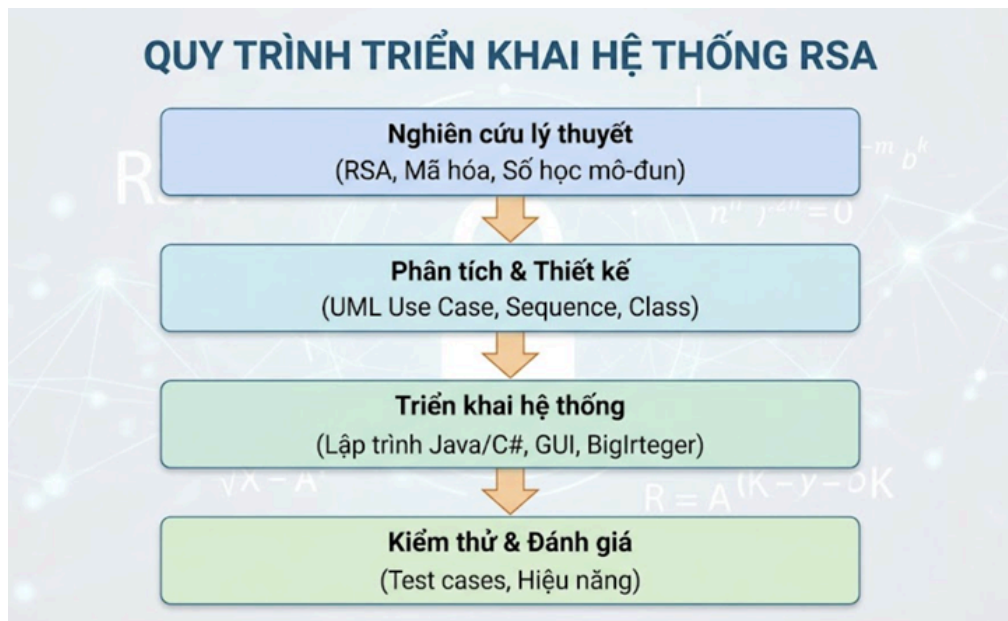
Đề tài có đề cập đến độ an toàn của RSA, tuy nhiên phạm vi phân tích chủ yếu dừng ở mức cơ sở lý thuyết. Nội dung nghiên cứu tập trung vào việc giải thích vì sao RSA có độ an toàn dựa trên độ khó của bài toán phân tích một số nguyên lớn thành tích của hai số nguyên tố. Một số nguy cơ tấn công phổ biến như tấn công vét cạn và tấn công phân tích thừa số nguyên tố được xem xét ở mức tổng quan để thấy được vai trò của việc lựa chọn khóa đủ lớn và bảo vệ khóa bí mật.

Trong phạm vi đề tài này, nhóm không đi sâu xây dựng một hệ thống bảo mật hoàn chỉnh dùng trong thực tế như hạ tầng khóa công khai, chứng chỉ số, giao thức truyền thông an toàn hoặc hệ thống chữ ký số đầy đủ. Các nội dung như DES, AES, Diffie-Hellman, SSL/TLS, PGP hoặc S/MIME chỉ được xem là kiến thức liên quan nếu cần so sánh hoặc mở rộng, không phải trọng tâm nghiên cứu chính của đề tài.

Tóm lại, phạm vi nghiên cứu của đề tài được giới hạn trong việc tìm hiểu nguyên lý hoạt động của hệ mật RSA, trình bày quy trình tạo khóa, mã hóa và giải mã, phân tích cơ sở an toàn của thuật toán ở mức cơ bản, đồng thời xây dựng chương trình minh họa bằng Java và C#. Đây là phạm vi phù hợp với mục tiêu học phần, giúp nhóm vận dụng kiến thức an toàn bảo mật thông tin vào một bài toán lập trình cụ thể.

1.3.4. Phương pháp nghiên cứu và triển khai hệ thống:

Để xây dựng chương trình mã hóa và giải mã RSA, nhóm đã áp dụng phương pháp nghiên cứu kết hợp giữa lý thuyết mật mã học và quy trình phát triển phần mềm nhằm đảm bảo hệ thống có tính chính xác, tính ổn định và khả năng hoạt động thực tế. Quá trình thực hiện được triển khai theo nhiều giai đoạn từ nghiên cứu thuật toán, phân tích thiết kế hệ thống đến cài đặt và kiểm thử chương trình.



Hình 1.x: Quy trình triển khai hệ thống RSA

1. Phương pháp nghiên cứu lý thuyết

Trong giai đoạn đầu, nhóm tiến hành tìm hiểu các kiến thức nền tảng liên quan đến lĩnh vực an toàn và bảo mật thông tin, đặc biệt là hệ mật mã khóa công khai RSA. Các tài liệu được tham khảo bao gồm giáo trình mật mã học, tài liệu học thuật về RSA và các nguồn tham khảo liên quan đến lập trình bảo mật. Quá trình nghiên cứu tập trung vào các nội dung chính như:

- Khái niệm về mã hóa và giải mã dữ liệu
- Nguyên lý hoạt động của hệ mật mã khóa công khai
- Cơ chế sinh khóa trong thuật toán RSA
- Quy trình mã hóa và giải mã dữ liệu
- Vai trò của số nguyên tố lớn trong bảo mật thông tin

Ngoài ra, nhóm cũng nghiên cứu các cơ sở toán học được sử dụng trong RSA nhằm đảm bảo tính chính xác khi triển khai thuật toán trên chương trình thực

tế. Các kiến thức toán học được sử dụng gồm: Phép toán modulo, Hàm Euler, Thuật toán Euclid mở rộng, Lũy thừa mô-đun, Nghịch đảo modulo.

Trong thuật toán RSA, hai số nguyên tố lớn p và q được lựa chọn để tạo khóa. Sau đó tính giá trị:

$$n = p \times q$$

Tiếp theo tính hàm Euler:

$$\varphi(n) = (p - 1)(q - 1)$$

Sau khi lựa chọn khóa công khai e , khóa bí mật d được xác định sao cho:

$$d \times e \equiv 1(\text{mod}\varphi(n))$$

Quá trình mã hóa bản rõ M thành bản mã C được thực hiện theo công thức:

$$C = M^e \text{ mod } n$$

Ngược lại, quá trình giải mã được thực hiện bằng công thức:

$$M = C^d \text{ mod } n$$

Việc nghiên cứu kỹ các công thức toán học giúp nhóm hiểu rõ nguyên lý hoạt động của RSA trước khi tiến hành xây dựng chương trình.

2.Phương pháp phân tích và thiết kế hệ thống

Sau khi hoàn thành quá trình nghiên cứu lý thuyết, nhóm tiến hành phân tích và thiết kế hệ thống theo hướng lập trình hướng đối tượng nhằm giúp chương trình có cấu trúc rõ ràng, dễ quản lý và thuận tiện trong việc mở rộng sau này.

Nhóm sử dụng phương pháp phân tích và thiết kế hướng đối tượng (OOAD) kết hợp với ngôn ngữ mô hình hóa UML để mô tả cấu trúc và hoạt động của hệ thống.

Các biểu đồ UML được sử dụng bao gồm:

- Use Case Diagram:** mô tả các chức năng chính của hệ thống và sự tương tác giữa người dùng với chương trình.

- Sequence Diagram:** mô tả trình tự xử lý dữ liệu trong quá trình sinh khóa, mã hóa và giải mã.

-Class Diagram: mô tả cấu trúc các lớp, thuộc tính và phương thức của hệ thống.

Hệ thống được chia thành các chức năng chính:

- Sinh khóa RSA
- Nhập dữ liệu cần mã hóa
- Thực hiện mã hóa dữ liệu
- Thực hiện giải mã dữ liệu
- Hiển thị kết quả đầu ra

Trong quá trình thiết kế, nhóm áp dụng nguyên tắc mô-đun hóa nhằm tách biệt phần giao diện người dùng với phần xử lý thuật toán. Điều này giúp chương trình dễ bảo trì, giảm lỗi và nâng cao khả năng tái sử dụng mã nguồn.

3. Phương pháp triển khai hệ thống:

Sau giai đoạn thiết kế, nhóm tiến hành xây dựng chương trình bằng ngôn ngữ Java/C#. Chương trình được phát triển theo mô hình tuần tự, bao gồm các bước từ xây dựng giao diện, xử lý thuật toán đến kiểm tra và hoàn thiện hệ thống.

Trong quá trình lập trình, nhóm sử dụng thư viện hỗ trợ xử lý số nguyên lớn nhằm thực hiện các phép toán với khóa RSA có kích thước lớn. Điều này giúp đảm bảo tính chính xác của các phép toán modulo và lũy thừa mô-đun trong quá trình mã hóa và giải mã.

Các chức năng chính được triển khai gồm:

- Chức năng sinh khóa công khai và khóa bí mật
- Chức năng mã hóa dữ liệu
- Chức năng giải mã dữ liệu
- Chức năng nhập và xuất dữ liệu
- Chức năng hiển thị kết quả trên giao diện chương trình

Ngoài ra, nhóm cũng xử lý các trường hợp ngoại lệ như:

- Dữ liệu đầu vào rỗng
- Khóa không hợp lệ
- Dữ liệu vượt quá giới hạn cho phép
- Lỗi trong quá trình mã hóa hoặc giải mã

Việc xử lý ngoại lệ giúp chương trình hoạt động ổn định và hạn chế lỗi trong quá trình sử dụng thực tế.

4. Phương pháp kiểm thử và đánh giá hệ thống Sau khi hoàn thiện chương trình, nhóm tiến hành kiểm thử nhằm đánh giá tính chính xác và khả năng hoạt động của hệ thống.

Nhóm sử dụng phương pháp kiểm thử chức năng (Functional Testing) để kiểm tra các chức năng chính của chương trình như: Sinh khóa RSA, Mã hóa dữ liệu, Giải mã dữ liệu, Hiển thị kết quả.

Các bộ dữ liệu thử nghiệm bao gồm:

- Chuỗi văn bản ngắn
- Đoạn văn bản dài
- Ký tự đặc biệt
- Dữ liệu có dấu tiếng Việt

Kết quả kiểm thử cho thấy dữ liệu sau khi giải mã có thể khôi phục chính xác nội dung ban đầu, chứng minh tính đúng đắn của thuật toán RSA khi triển khai trong chương trình.

Ngoài ra, nhóm cũng tiến hành đánh giá hiệu năng hệ thống thông qua việc đo thời gian thực hiện các quá trình: Sinh khóa, Mã hóa, Giải mã ở nhiều kích thước khóa khác nhau nhằm đánh giá ảnh hưởng của độ dài khóa đến tốc độ xử lý của hệ thống.

CHƯƠNG 2: KẾT QUẢ NGHIÊN CỨU

2.1. Nghiên cứu hệ mã hóa khóa công khai

2.1.1. Nguyên lý hoạt động của hệ mật mã khóa công khai.

Hệ mật mã khóa công khai (Public Key Cryptography) là phương pháp mã hóa sử dụng hai loại khóa khác nhau nhưng có mối liên hệ toán học chặt chẽ với nhau, bao gồm khóa công khai và khóa bí mật. Đây là một bước tiến quan trọng trong lĩnh vực an toàn thông tin vì giúp giải quyết được vấn đề phân phối khóa tồn tại ở các hệ mật mã truyền thống.

Trong hệ mật mã này, mỗi người dùng sẽ sở hữu một cặp khóa:

- Khóa công khai (Public Key): được công bố rộng rãi cho mọi người sử dụng.

-Khóa bí mật (Private Key): chỉ chủ sở hữu biết và phải được giữ bí mật tuyệt đối.

Nguyên lý hoạt động của hệ mật mã khóa công khai dựa trên việc dữ liệu được mã hóa bằng một khóa nhưng chỉ có thể giải mã bằng khóa còn lại trong cùng cặp khóa. Điều này giúp đảm bảo tính bảo mật của thông tin khi truyền qua môi trường mạng.

Quy trình hoạt động:

Bước 1: Sinh cặp khóa

- Trước tiên, người dùng tạo ra một cặp khóa gồm khóa công khai (K_p) và khóa bí mật (K_s) thông qua thuật toán sinh khóa.
- Khóa công khai (K_p) được chia sẻ cho mọi người.
- Khóa bí mật (K_s) được lưu giữ riêng và không được tiết lộ.
- Hai khóa này có quan hệ toán học với nhau nhưng rất khó suy ra khóa bí mật từ khóa công khai.

Bước 2: Mã hóa thông tin

- Khi người gửi muốn gửi dữ liệu bí mật (bản rõ M) cho người nhận, người gửi sẽ sử dụng khóa công khai (K_p) của người nhận để mã hóa thông điệp.
- Kết quả sau quá trình mã hóa là một bản mã C mà người ngoài không thể đọc hiểu được nếu không có khóa giải mã tương ứng.
- Biểu thức mã hoá: $C = E(K_p, M)$. Trong đó: E - thuật toán mã hoá
 K_p - khoá công khai

Bước 3: Giải mã thông tin

- Sau khi nhận được bản mã C , người nhận sử dụng khóa bí mật (K_s) của mình để giải mã và khôi phục lại nội dung ban đầu.
- Biểu thức giải mã: $M = D(K_s, C)$. Trong đó: D - thuật toán giải mã
 K_s - khoá bí mật
- Chỉ khóa bí mật tương ứng mới có thể giải mã đúng dữ liệu đã được mã hóa bằng khóa công khai.

Minh họa nguyên lý hoạt động:

1. Giả sử người A muốn gửi thông tin bí mật cho người B
2. Người B công khai khóa công khai của mình.
3. Người A sử dụng khóa công khai đó để mã hóa thông điệp.
4. Bản mã được gửi qua mạng Internet.
5. Người B sử dụng khóa bí mật để giải mã và đọc nội dung thông điệp.

Trong quá trình truyền dữ liệu, dù kẻ tấn công có chặn được bản mã và khóa công khai thì cũng rất khó giải mã thông tin nếu không có khóa bí mật tương ứng.

2.1.2. Quy trình trao đổi khóa và vấn đề phân phối khóa.

1. Quy trình trao đổi khóa:

Mã hoá khoá công khai thường được sử dụng để trao đổi khóa phiên, sau đó dữ liệu sẽ được mã hoá bằng thuật toán đối xứng để tăng tốc độ xử lý.

Quy trình trao đổi khóa được thực hiện như sau:

Bước 1: Tạo cặp khóa

Mỗi người dùng tự thiết lập một cặp khóa bất đối xứng bao gồm: khóa công khai (Public key) công bố rộng rãi và khóa bí mật (Private key) được lưu trữ tuyệt mật tại thiết bị đầu cuối.

Bước 2: Công bố và xác thực khóa công khai

Người dùng công bố khóa công khai của mình thông qua các môi trường lưu trữ (như máy chủ, website hoặc đính kèm trong chứng thư số). Trước khi tiến hành giao tiếp, người gửi phải thực hiện quy trình xác minh đảm bảo khóa công khai này thực sự thuộc về người nhận hợp pháp tránh nguy cơ giả mạo.

Bước 3: Khởi tạo khóa phiên

Sau khi xác thực thành công, người gửi tạo ra một khóa phiên (Session key) ngẫu nhiên. Khóa này tuân theo các thuật toán mã hoá đối xứng và chỉ có giá trị sử dụng trong một phiên làm việc nhất định để hạn chế rủi ro bị giải mã hồi tố.

Bước 4: Mã hoá khóa phiên

Người gửi sử dụng khóa công khai đã được xác thực của người nhận để mã hoá khóa phiên, chuyển đổi khóa phiên thành dạng mã an toàn. Sau đó, bản mã này được truyền qua môi trường mạng đến người nhận.

Bước 5: Giải mã khóa phiên

Người nhận tiếp nhận bản mã chứa khóa phiên, sử dụng khóa bí mật tương ứng của mình để giải mã. Lúc này, cả hai bên đều sở hữu cùng một khóa phiên mà không để lộ thông tin ra ngoài mạng.

Bước 6: Trao đổi dữ liệu

Sau khi hoàn tất quá trình trao đổi khóa, hai bên sử dụng khóa phiên chung kết hợp với các thuật toán mã hoá đối xứng để tiến hành truyền nhận dữ liệu an toàn.

2.Vấn đề phân phối khoá

Mặc dù mã hóa khóa công khai (Asymmetric Cryptography) giúp khắc phục hạn chế của mã hóa đối xứng trong việc trao đổi khóa bí mật thông qua một kênh truyền thông an toàn. Tuy nhiên, hệ thống vẫn tồn tại vấn đề quan trọng là bài toán xác định thực thể và tính toàn vẹn của khóa công khai.

Do khóa công khai được công bố rộng rãi nên kẻ tấn công có thể thực hiện hành vi giả mạo khóa nhằm đánh cắp thông tin trong quá trình truyền dữ liệu. Nếu người gửi sử dụng nhầm khóa công khai giả mạo, dữ liệu mã hóa có thể bị giải mã bởi đối tượng tấn công thay vì người nhận hợp pháp.

Ví dụ:

1. Người dùng A muốn gửi dữ liệu cho người dùng B.
2. A yêu cầu khóa công khai của B.
3. Kẻ tấn công gửi cho A một khóa công khai giả mạo.
4. A sử dụng khóa này để mã hóa dữ liệu.
5. Kẻ tấn công dùng khóa bí mật tương ứng để giải mã và đọc thông tin.

Hệ quả là kênh truyền bị thỏa hiệp hoàn toàn mà hai thực thể hợp pháp không hề hay biết. Do đó, việc thiết lập một cơ chế xác minh tính chính xác và nguồn gốc của khóa công khai là yêu cầu rất quan trọng trong an toàn thông tin.

Các phương pháp phân phối khóa công khai hiện nay:

Công bố công khai: Khóa công khai được đăng tải trực tiếp lên các kênh truyền thông công cộng.

-Ưu điểm: Đơn giản, chi phí triển khai thấp và không phụ thuộc vào bên thứ ba.

-Nhược điểm: Mức độ an toàn thấp. Kẻ tấn công dễ dàng giả mạo danh hoặc thay đổi gián tiếp các liên kết tải khoá trên giao diện web công cộng.

Máy chủ khóa công khai: Hệ thống sử dụng một máy chủ trung tâm chuyên dụng (Authority) chịu trách nhiệm lưu trữ và danh bạ hoá toàn bộ khóa công khai của các người dùng trong mạng.

-Ưu điểm: Quản lý tập trung, hỗ trợ cơ chế cập nhật và thu hồi khóa linh hoạt hơn so với công bố tự do.

-Nhược điểm: Nếu máy chủ bị tấn công từ chối dịch vụ (DoS) hoặc bị chiếm quyền kiểm soát, toàn bộ mạng lưới phân phối khoá sẽ bị sụp đổ hoặc bị thao túng dữ liệu.

Chứng thư số: Chứng thư là phương pháp phổ biến và an toàn nhất hiện nay. Phương pháp này sử dụng một bên thứ ba độc lập được tin tưởng tuyệt đối gọi là Tổ chức chứng thực (Certificate Authority-CA). Chứng thư số chứa:

- Thông tin chủ sở hữu
- Khoá công khai
- Chữ ký xác nhận của CA

Nhờ đó người dùng có thể xác minh chính xác khoá công khai thuộc về ai và hạn chế nguy cơ giả mạo.

2.1.3. So sánh ưu, nhược điểm giữa mã hóa khóa công khai và khóa bí mật.

So sánh ưu, nhược điểm mã hoá khóa công khai và khóa bí mật

Mã hoá khóa bí mật:

- Ưu điểm:
 - + Tốc độ mã hoá và giải mã nhanh do thuật toán có độ phức tạp thấp.
 - + Hiệu suất cao, phù hợp để mã hoá dữ liệu lớn.
 - + Tiêu tốn ít tài nguyên tính toán và bộ nhớ lớn.
 - + Dễ triển khai trong các hệ thống yêu cầu xử lý dữ liệu nhanh.
- Nhược điểm:
 - + Gặp khó khăn trong việc phân phối khoá giữa các bên tham gia liên lạc.
 - + Nếu khoá bí mật bị lộ, toàn bộ dữ liệu được mã hoá bằng khoá đó đều có nguy cơ bị giải mã.
 - + Số lượng khoá cần quản lý tăng nhanh khi số lượng người dùng tăng.
 - + Không hỗ trợ xác thực danh tính và chữ ký số.

Mã hoá khóa công khai:

- Ưu điểm:
 - + Giải quyết được vấn đề phân phối khoá vì khóa công khai có thể công khai cho mọi người.
 - + Tăng cường tính bảo mật trong quá trình trao đổi thông tin.
 - + Hỗ trợ xác thực người dùng và chữ ký số.

- + Đảm bảo tính xác thực, toàn vẹn và chống chối bỏ thông tin.
- Nhược điểm:
 - + Tốc độ mã hoá và giải mã chậm hơn so với mã hoá bí mật.
 - + Yêu cầu nhiều tài nguyên xử lý và bộ nhớ hơn.
 - + Không phù hợp để mã hoá trực tiếp lượng dữ liệu lớn.
 - + Việc tạo và quản lý cặp khoá phức tạp hơn.

Bảng so sánh:

Tiêu chí	Mã hoá khoá bí mật	Mã hoá khoá công khai
Số lượng khoá	1 khoá (dùng chung)	1 cặp khoá (công khai và bí mật)
Tốc độ xử lý	Nhanh	Chậm hơn
Hiệu suất	Cao	Thấp hơn
Phân phối khoá	Khó khăn	Dễ dàng
Quản lý khoá	Phức tạp khi nhiều người dùng	Đơn giản hơn
Chữ ký số	Không hỗ trợ	Có hỗ trợ
Xác thực người dùng	Hạn chế	Tốt
Ứng dụng	Mã hoá dữ liệu	Trao đổi khoá, chữ ký số

2.2. Tìm hiểu chi tiết thuật toán RSA

2.2.1. Cấu trúc của thuật toán RSA:

2.2.2. Quy trình thuật toán:

1. Giai đoạn tạo cặp khóa Public Key và Private Key

Giai đoạn tạo cặp khóa là bước đầu tiên trong thuật toán RSA. Mục đích của giai đoạn này là tạo ra hai khóa khác nhau nhưng có quan hệ toán học với nhau, gồm khóa công khai và khóa bí mật. Khóa công khai được sử dụng để mã hóa dữ liệu hoặc kiểm tra chữ ký số, còn khóa bí mật được sử dụng để giải mã dữ liệu hoặc tạo chữ ký số.

Quy trình tạo khóa RSA được thực hiện qua các bước sau:

Bước 1: Chọn hai số nguyên tố lớn p và q

- Chọn hai số nguyên tố lớn khác nhau, ký hiệu là: p và q
- Điều kiện: $p \neq q$

Hai số nguyên tố này phải được chọn đủ lớn để bảo đảm độ an toàn cho hệ mật RSA. Trong thực tế, nếu p và q quá nhỏ thì kẻ tấn công có thể dễ dàng phân tích n thành tích của hai số nguyên tố, từ đó suy ra khóa bí mật.

Bước 2: Tính giá trị n

- Sau khi chọn được p và q, ta tính: $n = p \times q$

Giá trị n là tích của hai số nguyên tố p và q. Đây là thành phần chung xuất hiện trong cả khóa công khai và khóa bí mật. Giá trị n cũng được dùng làm modulo trong quá trình mã hóa và giải mã RSA.

Bước 3: Tính giá trị $\varphi(n)$

- Tiếp theo, tính hàm Euler của n theo công thức:

$$\varphi(n) = (p - 1) \times (q - 1)$$

Giá trị $\varphi(n)$ có vai trò quan trọng trong việc chọn số mũ công khai e và tính số mũ bí mật d.

Bước 4: Chọn số nguyên e

- Chọn một số nguyên e sao cho e nguyên tố cùng nhau với $\varphi(n)$. Có thể hiểu là ước chung lớn nhất của e và $\varphi(n)$ phải bằng 1: $\gcd(e, \varphi(n)) = 1$
- Thông thường, e được chọn sao cho: $1 < e < \varphi(n)$

Giá trị e sẽ được sử dụng làm số mũ công khai trong khóa công khai. Điều kiện $\gcd(e, \varphi(n)) = 1$ giúp bảo đảm rằng có thể tìm được nghịch đảo modulo của e.

Bước 5: Tính số nguyên d

- Tính d là nghịch đảo modulo của e theo $\varphi(n)$, theo công thức: $d = e^{-1} \bmod \varphi(n)$

- Điều đó có nghĩa là d phải thỏa mãn: $e \times d \equiv 1 \bmod \varphi(n)$

Giá trị d là số mũ bí mật, được sử dụng trong quá trình giải mã. Đây là thành phần cần được bảo vệ tuyệt đối, không được công khai cho người khác biết.

Bước 6: Xác định cặp khóa RSA

Sau khi tính được các giá trị cần thiết, ta xác định cặp khóa như sau:

- **Khóa công khai:** Public Key = $\{e, n\}$ hoặc ký hiệu: $K_p = \{e, n\}$
- **Khóa bí mật:** Private Key = $\{d, n\}$ hoặc ký hiệu: $K_s = \{d, n\}$

Trong đó, khóa công khai $\{e, n\}$ có thể được công bố để người khác sử dụng khi mã hóa dữ liệu gửi đến chủ sở hữu khóa. Ngược lại, khóa bí mật $\{d, n\}$ phải được giữ kín vì khóa này dùng để giải mã dữ liệu đã được mã hóa bằng khóa công khai tương ứng.

Bước 7: Loại bỏ hoặc bảo vệ các giá trị trung gian

Sau khi tạo khóa, các giá trị p , q và $\phi(n)$ cần được bảo vệ cẩn thận hoặc không lưu trữ công khai. Nguyên nhân là nếu kẻ tấn công biết được p và q , họ có thể tính được: $\phi(n) = (p - 1) \times (q - 1)$

Từ đó, họ có thể suy ra d và phá vỡ hệ mật RSA. Vì vậy, trong hệ RSA, độ an toàn phụ thuộc rất lớn vào việc giữ bí mật các giá trị này và lựa chọn p , q đủ lớn.

Tóm lại, giai đoạn tạo cặp khóa của RSA bắt đầu từ việc chọn hai số nguyên tố lớn p và q , sau đó tính n , $\phi(n)$, chọn e , tính d , rồi tạo ra hai khóa: khóa công khai $\{e, n\}$ và khóa bí mật $\{d, n\}$. Đây là nền tảng để thực hiện các giai đoạn tiếp theo của thuật toán RSA là mã hóa và giải mã.

2. Giai đoạn mã hóa

Giai đoạn mã hóa trong thuật toán RSA là quá trình chuyển đổi bản rõ (Plaintext - thông điệp ban đầu) thành bản mã (Ciphertext - dữ liệu không thể đọc hiểu) bằng khóa công khai của người nhận. Quá trình này đảm bảo tính bảo mật tuyệt đối cho dữ liệu khi truyền tải qua môi trường mạng.

Thuật toán mã hóa RSA

-Giả sử:

- + Bản rõ sau khi xử lý là số nguyên M (với điều kiện $0 \leq M < n$).
- + Khóa công khai dùng để mã hóa là cặp (e, n) .

+ Bản mã sau quá trình tính toán là C.

-Thuật toán mã hóa RSA được thực hiện theo công thức:

$$C = M^e \bmod n$$

Các bước của thuật toán mã hóa RSA

-Bước 1: Nhập và tiền xử lý bản rõ

+ Người gửi tiếp nhận thông điệp (văn bản) cần truyền đi.

+Chuyển đổi chuỗi văn bản thành định dạng byte (thông qua bảng mã ASCII/Unicode).

-Bước 2: Sử dụng khóa công khai

+ Người gửi sử dụng khóa công khai (e, n) của người nhận để chuẩn bị mã hóa.

+ Khóa này được công bố rộng rãi trên mạng và bất kỳ ai cũng có thể sử dụng.

-Bước 3: Thực hiện phép toán mã hóa: Áp dụng công thức tính bản mã C.

-Bước 4: Xuất bản mã

+ Kết quả sau phép tính toán học là bản mã C.

+ Bản mã này đã an toàn và sẵn sàng để gửi đi qua môi trường mạng Internet.

Mô tả thuật toán mã hóa RSA

- Input:

+ Bản rõ (Thông điệp gốc cần mã hóa).

+ Khóa công khai (e, n)

- Output:

+ Bản mã C

Các bước thực hiện

1. Nhận giá trị bản rõ M.

2. Nhập khóa công khai (e, n).

3. Tính toán: $C = M^e \bmod n$
4. Xuất ra bản mã C để truyền đi.

3. Giai đoạn giải mã

Giai đoạn giải mã trong thuật toán RSA là quá trình chuyển đổi bản mã (Ciphertext) trở lại thành bản rõ (Plaintext) bằng khóa bí mật của người nhận. Quá trình này đảm bảo rằng chỉ người sở hữu khóa bí mật tương ứng mới có thể đọc được nội dung thông tin đã mã hóa.

Thuật toán giải mã RSA

- Giả sử:

- + Bản mã là C
- + Khóa bí mật là (d, n)
- + Bản rõ sau giải mã là M
- Thuật toán giải mã RSA được thực hiện theo công thức: $M = C^d \bmod n$.

Các bước của thuật toán giải mã RSA

- Bước 1: Nhập bản mã

- + Người nhận tiếp nhận bản mã C đã được gửi từ bên gửi.
- + Bản mã này được tạo ra từ quá trình mã hóa bằng khóa công khai.

- Bước 2: Sử dụng khóa bí mật

- + Người nhận sử dụng khóa bí mật (d, n) để thực hiện phép giải mã.
- + Khóa bí mật chỉ được lưu giữ bởi người nhận nhằm đảm bảo tính an toàn của dữ liệu.

- Bước 3: Thực hiện phép toán giải mã

- + Áp dụng công thức: $M = C^d \bmod n$.

- Bước 4: Xuất bản rõ

- + Kết quả sau phép tính là bản rõ M , chính là thông điệp ban đầu trước khi mã hóa.
- + Người nhận có thể đọc và sử dụng dữ liệu này như dữ liệu gốc.

Mô tả thuật toán giải mã RSA

- Input:

- + Bản mã C
- + Khóa bí mật (d, n)

- Output:

- + Bản rõ M

Các bước thực hiện

1. Nhận giá trị bản mã C

2. Nhập khóa bí mật (d,n)

3. Tính:

- $M = C^d \bmod n$.

- Xuất ra bản rõ M

2.2.3. Ưu và nhược điểm của thuật toán RSA.

2.2.4. Các cuộc tấn công phổ biến vào RSA và cách phòng chống

RSA là một hệ mật mã khóa công khai có độ an toàn phụ thuộc nhiều vào việc lựa chọn tham số khóa và khả năng bảo vệ khóa bí mật. Trong RSA, khóa công khai có dạng $\{e, n\}$, khóa bí mật có dạng $\{d, n\}$. Giá trị n được tạo ra từ tích của hai số nguyên tố lớn p và q:

$$n = p \times q$$

Khi biết được p và q, ta có thể tính được:

$$\varphi(n) = (p - 1) \times (q - 1)$$

Từ đó có thể suy ra khóa bí mật d theo công thức:

$$d = e^{-1} \bmod \varphi(n)$$

Vì vậy, nếu kẻ tấn công tìm được d hoặc phân tích được n để tìm ra p và q, hệ mật RSA có thể bị phá vỡ. Trong phạm vi đề tài, hai dạng tấn công phổ biến được xét đến là tấn công vét cạn và tấn công phân tích thừa số nguyên tố.

1. Tấn công vét cạn

Tấn công vét cạn là phương pháp tấn công trong đó kẻ tấn công thử lần lượt nhiều khả năng khác nhau để tìm ra khóa bí mật hoặc thông điệp ban đầu. Đối với hệ mật RSA, kẻ tấn công có thể cố gắng thử các giá trị có khả năng là khóa bí mật d, hoặc thử các khả năng của bản rõ M để tìm ra thông điệp phù hợp với bản mã đã có.

Trong quá trình mã hóa RSA, bản rõ M được biến đổi thành bản mã C theo công thức:

$$C = M^e \bmod n$$

Để khôi phục lại thông điệp ban đầu, cần sử dụng khóa bí mật d:

$$M = C^d \bmod n$$

Nếu không biết d , kẻ tấn công có thể tìm cách thử nhiều giá trị khác nhau cho đến khi tìm được kết quả đúng. Tuy nhiên, phương pháp này chỉ có khả năng thực hiện được khi không gian khóa nhỏ, tức là các tham số trong RSA được chọn quá nhỏ hoặc không đủ an toàn. Khi p , q , n và d có kích thước lớn, số lượng trường hợp cần thử sẽ rất lớn, khiến việc vét cạn gần như không khả thi trong thời gian tính toán hợp lý.

Ví dụ, nếu trong chương trình minh họa chỉ sử dụng các số nguyên tố nhỏ như $p = 11$, $q = 17$ thì việc thử các giá trị để tìm khóa có thể thực hiện khá dễ dàng. Tuy nhiên, trong thực tế, các số nguyên tố được sử dụng trong RSA phải có kích thước lớn hơn rất nhiều để làm tăng độ khó cho quá trình tấn công.

Cách phòng chống tấn công vét cạn:

Để hạn chế nguy cơ bị tấn công vét cạn, cần lựa chọn các tham số RSA đủ lớn. Hai số nguyên tố p và q không nên chọn quá nhỏ, vì nếu p và q nhỏ thì $n = p \times q$ cũng nhỏ, làm cho không gian khóa bị thu hẹp. Khi không gian khóa nhỏ, kẻ tấn công có thể thử lần lượt các khả năng để tìm khóa bí mật.

Ngoài ra, cần chọn p và q một cách ngẫu nhiên, bảo đảm chúng là hai số nguyên tố khác nhau và không dễ đoán. Khóa bí mật d phải được bảo vệ cẩn thận, không lưu trữ hoặc truyền tải dưới dạng dễ bị đọc trộm. Đối với chương trình cài đặt bằng Java và C#, cần tránh sử dụng các giá trị khóa quá nhỏ ngoài mục đích minh họa thuật toán.

2. Tấn công phân tích thừa số nguyên tố

Tấn công phân tích thừa số nguyên tố là dạng tấn công quan trọng nhất đối với RSA. Bản chất của phương pháp này là kẻ tấn công tìm cách phân tích giá trị công khai n thành tích của hai số nguyên tố ban đầu p và q .

Trong RSA, giá trị n được công khai cùng với e trong khóa công khai:

$$\text{Public Key} = \{e, n\}$$

Do đó, bất kỳ ai cũng có thể biết được n . Tuy nhiên, độ an toàn của RSA nằm ở chỗ: mặc dù biết n , việc tìm lại p và q từ công thức:

$$n = p \times q$$

là rất khó nếu p và q là các số nguyên tố đủ lớn.

Nếu kẻ tấn công phân tích được n thành p và q , họ có thể tính được:

$$\varphi(n) = (p - 1) \times (q - 1)$$

Sau khi biết được $\varphi(n)$, kẻ tấn công có thể tính khóa bí mật:

$$d = e^{-1} \bmod \varphi(n)$$

Khi đã có d , kẻ tấn công có thể giải mã bản mã C để thu được thông điệp gốc M :

$$M = C^d \bmod n$$

Như vậy, việc phân tích được n thành p và q đồng nghĩa với việc phá vỡ khóa bí mật của hệ RSA. Đây là lý do tại sao việc lựa chọn hai số nguyên tố lớn p và q có vai trò đặc biệt quan trọng trong quá trình tạo khóa RSA.

Cách phòng chống tấn công phân tích thừa số nguyên tố:

Để phòng chống dạng tấn công này, cần chọn hai số nguyên tố p và q có kích thước đủ lớn. Khi p và q càng lớn, giá trị $n = p \times q$ càng lớn, làm cho việc phân tích n thành hai thừa số nguyên tố trở nên khó khăn hơn rất nhiều về mặt tính toán.

Bên cạnh đó, không nên chọn p và q quá gần nhau hoặc có quy luật dễ đoán. Nếu hai số nguyên tố được chọn không an toàn, kẻ tấn công có thể lợi dụng đặc điểm này để rút ngắn thời gian phân tích n . Do đó, trong quá trình sinh khóa RSA, cần chọn p và q ngẫu nhiên, đủ lớn và khác nhau.

Trong chương trình thực nghiệm, có thể dùng các số nguyên tố nhỏ để minh họa rõ các bước sinh khóa, mã hóa và giải mã. Tuy nhiên, cần phân biệt rõ rằng các giá trị nhỏ chỉ phù hợp cho mục đích học tập. Nếu áp dụng vào thực tế, RSA phải sử dụng các số nguyên tố lớn để bảo đảm độ an toàn.

3. Nhận xét chung

Hai dạng tấn công trên đều hướng đến mục tiêu cuối cùng là tìm được khóa bí mật d hoặc khôi phục thông điệp gốc M . Tấn công vét cạn dựa trên việc thử nhiều khả năng khác nhau, còn tấn công phân tích thừa số nguyên tố tập trung vào việc phân tích n để tìm ra p và q .

Độ an toàn của RSA phụ thuộc trực tiếp vào kích thước khóa và cách lựa chọn tham số. Nếu các số nguyên tố p , q quá nhỏ, hệ mật dễ bị phá vỡ. Ngược lại, nếu p , q được chọn đủ lớn, ngẫu nhiên và khóa bí mật được bảo vệ tốt, việc tìm ra khóa bí mật từ khóa công khai sẽ rất khó thực hiện trong thời gian tính toán hợp lý.

Tóm lại, để tăng cường độ an toàn cho hệ mật RSA, cần thực hiện các biện pháp chính sau: chọn hai số nguyên tố lớn và khác nhau, tạo giá trị n đủ lớn, bảo vệ khóa bí mật d , không sử dụng khóa quá nhỏ trong các hệ thống thực tế và kiểm tra kỹ quá trình sinh khóa trong chương trình cài đặt. Đây là những yếu tố quan trọng giúp RSA hạn chế nguy cơ bị tấn công và bảo đảm hiệu quả trong mã hóa, giải mã dữ liệu.

2.3. Thiết kế chương trình và cài đặt thuật toán

2.3.1. Thiết kế kịch bản chương trình:

Chương trình RSA được phát triển nhằm minh họa nguyên lý hoạt động của hệ mật mã khóa công khai RSA trong việc bảo vệ dữ liệu. Thông qua chương trình, người dùng có thể thực hiện toàn bộ quy trình từ sinh khóa, mã hóa thông tin đến giải mã dữ liệu, qua đó hiểu rõ cách thức RSA sử dụng cặp khóa công khai và khóa bí mật để đảm bảo tính bảo mật của thông tin.

Bước 1: Khởi động chương trình

Người dùng chạy chương trình RSA. Sau khi khởi động, hệ thống hiển thị giao diện chính với các chức năng:

- Sinh khóa RSA.
- Mã hóa thông điệp.
- Giải mã thông điệp.
- Thoát chương trình.

Người dùng lựa chọn chức năng phù hợp để tiếp tục làm việc.

Bước 2: Sinh khóa RSA

Người dùng chọn chức năng sinh khóa và nhập hai số nguyên tố p và q khác nhau.

Tiến trình xử lý của hệ thống

1. Tính giá trị n theo công thức: $n = p \times q$
2. Tính giá trị hàm Euler: $\varphi(n) = (p - 1)(q - 1)$
3. Chọn số nguyên e sao cho: $1 < e < \varphi(n)$ và là nguyên tố cùng nhau

với $\gcd(e, \varphi(n)) = 1$

4. Tính số mũ bí mật d sao cho: $d \times e \equiv 1 \pmod{\varphi(n)}$

Kết quả

Hệ thống tạo ra:

-Khóa công khai $K_p = (e, n)$

-Khóa bí mật $K_s = (d, n)$

Các khóa được hiển thị trên giao diện để phục vụ quá trình mã hóa và giải mã.

Bước 3: Mã hóa dữ liệu

Người dùng chọn chức năng mã hóa và nhập bản rõ cần bảo vệ.

Tiến trình xử lý của hệ thống

Hệ thống sử dụng khóa công khai để thực hiện mã hóa theo công thức:

$$C = M^e \bmod n$$

Trong đó: M là bản rõ.

C là bản mã.

Kết quả

Bản mã sau khi mã hóa được hiển thị trên màn hình.

Bước 4: Giải mã dữ liệu

Người dùng chọn chức năng giải mã và nhập bản mã cần giải mã.

Tiến trình xử lý của hệ thống

Hệ thống sử dụng khóa bí mật để tính lại thông điệp gốc M từ thông điệp mã hoá C

$$M = C^d \bmod n$$

Kết quả: Bản rõ sau khi giải mã được hiển thị trên giao diện chương trình.

Bước 5: Kiểm tra kết quả

Sau khi giải mã, hệ thống so sánh bản rõ thu được với dữ liệu ban đầu.

Nếu dữ liệu sau giải mã trùng khớp với dữ liệu gốc thì quá trình mã hóa và giải mã được thực hiện thành công.

Ngược lại, hệ thống sẽ thông báo lỗi để người dùng kiểm tra lại khóa hoặc dữ liệu đầu vào.

Bước 6: Kết thúc chương trình

Người dùng lựa chọn chức năng "Thoát". Hệ thống đóng chương trình và kết thúc toàn bộ quá trình xử lý.

2.3.2. Giới thiệu ngôn ngữ lập trình sử dụng:

1. Giới thiệu ngôn ngữ lập trình Java

Java là ngôn ngữ lập trình hướng đối tượng được phát triển bởi Sun Microsystems và hiện nay thuộc Oracle Corporation. Java được sử dụng rộng rãi trong phát triển phần mềm nhờ tính bảo mật cao, khả năng hoạt động trên nhiều nền tảng khác nhau và hệ thống thư viện phong phú. Với đặc điểm "Write Once, Run Anywhere", các chương trình Java có thể thực thi trên nhiều hệ điều hành thông qua môi trường Java Virtual Machine (JVM).

Trong đề tài xây dựng chương trình mã hóa và giải mã RSA, Java là một lựa chọn phù hợp nhờ khả năng xử lý các phép toán trên số nguyên lớn và hỗ trợ nhiều thư viện liên quan đến bảo mật thông tin.

Các đặc điểm nổi bật của ngôn ngữ Java bao gồm:

- Hỗ trợ lập trình hướng đối tượng (OOP): giúp tổ chức chương trình theo các lớp và đối tượng, thuận lợi cho việc xây dựng các module sinh khóa, mã hóa và giải mã trong hệ mật RSA.
- Khả năng xử lý số nguyên lớn: Java cung cấp lớp BigInteger trong thư viện java.math, cho phép thực hiện các phép toán trên các số có kích thước rất lớn như sinh số nguyên tố, tính lũy thừa modulo và nghịch đảo modulo, đây là những phép toán cốt lõi của thuật toán RSA.
- Thư viện bảo mật phong phú: Java cung cấp nhiều thư viện hỗ trợ mật mã học và bảo mật dữ liệu, giúp việc xây dựng và kiểm thử chương trình RSA thuận tiện hơn.
- Tính độc lập nền tảng: chương trình RSA được phát triển bằng Java có thể chạy trên nhiều hệ điều hành khác nhau như Windows, Linux hoặc macOS mà không cần thay đổi mã nguồn.
- Quản lý bộ nhớ tự động: cơ chế Garbage Collection giúp quản lý bộ nhớ hiệu quả, giảm thiểu lỗi trong quá trình thực thi chương trình.

Với những ưu điểm trên, Java đáp ứng tốt các yêu cầu của đề tài xây dựng chương trình mã hóa và giải mã RSA. Đặc biệt, lớp BigInteger hỗ trợ hiệu quả các phép toán số học lớn, góp phần đơn giản hóa quá trình triển khai thuật toán và nâng cao độ chính xác của chương trình.

2. Giới thiệu ngôn ngữ lập trình C#

C# (đọc là C-Sharp) là một ngôn ngữ lập trình hướng đối tượng được phát triển bởi Microsoft và hoạt động trên nền tảng .NET. Đây là một ngôn ngữ hiện đại, đa năng, được sử dụng rộng rãi trong việc phát triển các ứng dụng desktop, web, di động cũng như các hệ thống phần mềm doanh nghiệp.

C# được thiết kế với mục tiêu kết hợp những ưu điểm của các ngôn ngữ lập trình trước đó như C++ và Java, đồng thời đơn giản hóa quá trình phát triển phần mềm. Nhờ cú pháp rõ ràng, dễ đọc và dễ bảo trì, C# đã trở thành một trong những ngôn ngữ lập trình phổ biến hiện nay.

Các đặc điểm nổi bật của ngôn ngữ C# bao gồm:

- Hỗ trợ lập trình hướng đối tượng (Object-Oriented Programming - OOP): C# hỗ trợ đầy đủ các đặc tính của lập trình hướng đối tượng như đóng gói, kế thừa và đa hình, giúp chương trình được tổ chức khoa học, dễ quản lý và mở rộng.
- An toàn kiểu dữ liệu (Type-safe): C# kiểm tra kiểu dữ liệu chặt chẽ trong quá trình biên dịch và thực thi, góp phần hạn chế các lỗi liên quan đến bộ nhớ và ép kiểu không hợp lệ.
- Quản lý bộ nhớ tự động: Thông qua cơ chế Garbage Collection của .NET, C# tự động thu hồi bộ nhớ không còn sử dụng, giúp giảm thiểu nguy cơ rò rỉ bộ nhớ và tăng độ ổn định cho ứng dụng.
- Thư viện .NET phong phú: Nền tảng .NET cung cấp hệ thống thư viện đa dạng hỗ trợ xử lý dữ liệu, làm việc với tệp tin, cơ sở dữ liệu, mạng máy tính, bảo mật và nhiều lĩnh vực khác.
- Hỗ trợ xây dựng giao diện đồ họa: C# cho phép phát triển các ứng dụng có giao diện trực quan thông qua các công nghệ như Windows Forms (WinForms) và Windows Presentation Foundation (WPF).

Trong đề tài xây dựng chương trình mã hóa và giải mã RSA, ngôn ngữ C# được lựa chọn làm công cụ phát triển chính nhờ những ưu điểm phù hợp với yêu cầu của hệ thống. C# cung cấp lớp BigInteger thuộc thư viện System.Numerics, cho phép thực hiện các phép toán trên số nguyên có kích thước rất lớn, đáp ứng yêu cầu tính toán của thuật toán RSA. Bên cạnh đó, các thư viện sẵn có của .NET giúp đơn giản hóa quá trình xây dựng chương trình, trong khi khả năng phát triển giao diện đồ họa hỗ trợ tạo ra một ứng dụng trực quan và thuận tiện cho người sử dụng.

Nhờ những đặc điểm trên, C# là một lựa chọn phù hợp để triển khai chương trình mã hóa và giải mã RSA trong phạm vi của đề tài.

2.3.3. Cài đặt thuật toán với ngôn ngữ JAVA:

2.3.4. Cài đặt thuật toán với ngôn ngữ C#: